

算法设计与分析

Design & Analysis of Algorithms

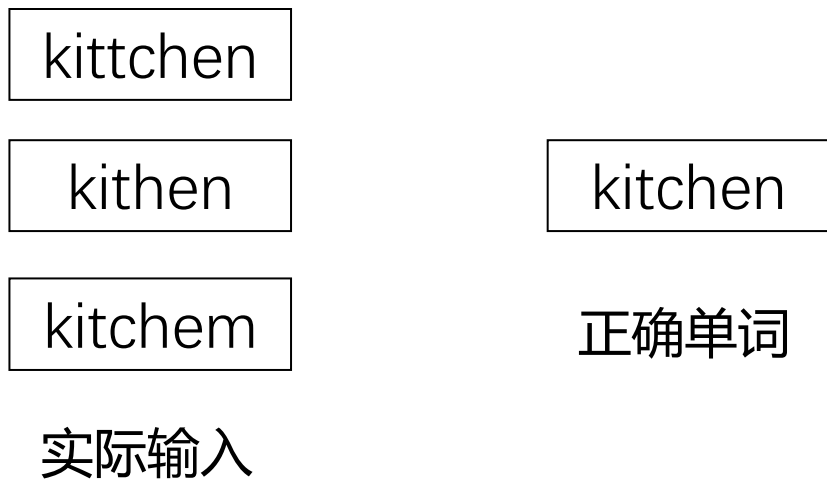
第3章 动态规划

学习要点

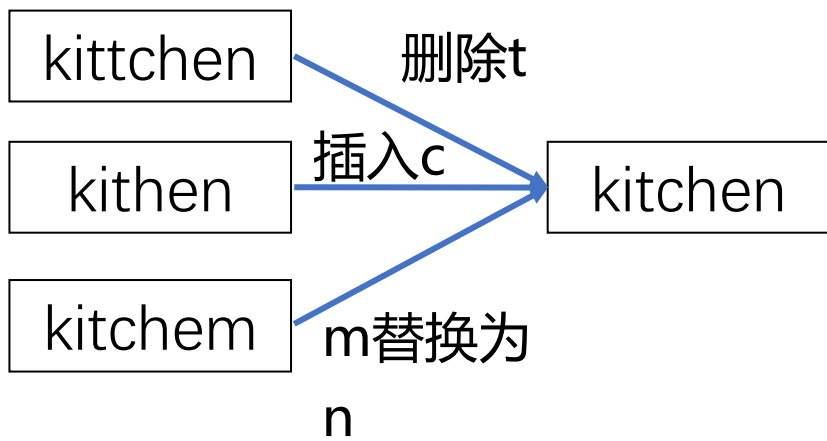
- 理解动态规划算法的概念
- 掌握动态规划算法的基本要素
 - (1) 最优子结构性质 (2) 重叠子问题性质
- 掌握设计动态规划算法的步骤
- 通过应用范例学习动态规划算法设计策略
 - (1) 基础题目
 - (2) 背包问题
 - (3) 子序列问题
 - (4) 图像压缩

3.3 子序列问题 编辑距离

- 问题背景 输入法自动更正



- 编辑操作实现自动更正 删除、插入、替换



3.3 子序列问题 编辑距离

- 编辑距离 最少的编辑操作数使两个串相等



- 问题描述

输入：长度为 n 的字符串 s , 长度为 m 的字符串 t

输出：求出一组编辑操作 $O = \langle e_1, e_2, \dots, e_d \rangle$, 令 $\min |O|$
使得字符串经过 O 的操作后满足 $s = t$.

3.3 子序列问题 编辑距离

- 给出问题表示

$D[i,j]$: 字符串 $s[1\dots i]$ 变成 $t[1\dots j]$ 的最小编辑距离

- 明确原始问题

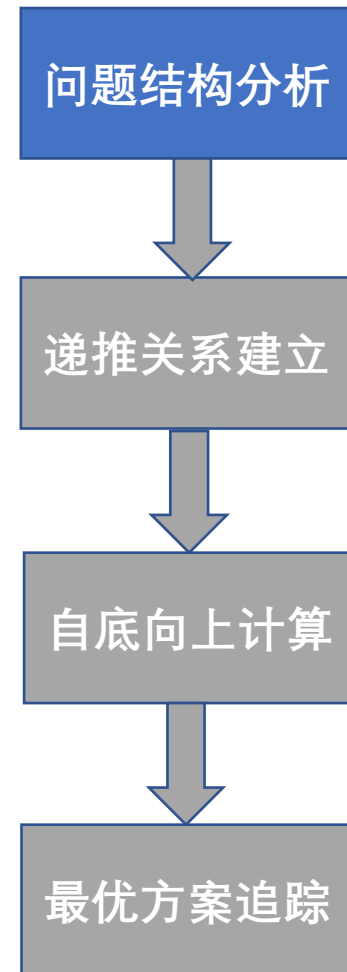
$D[n,m]$: 字符串 $s[1\dots n]$ 和 $t[1\dots m]$ 的最小编辑距离

问题结构分析

递推关系建立

自底向上计算

最优方案追踪



3.3 子序列问题 编辑距离

■ 考察末尾字符：删除

s A B C B D A **B**

t B D C A B A

s A B C B D A B

t B D C A B A

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.3 子序列问题 编辑距离

- 考察末尾字符 只操作s字符串

删除

s

A	B	C	B	D	A	B
---	---	---	---	---	---	---

t

B	D	C	A	B	A
---	---	---	---	---	---

插入

s

A	B	C	B	D	A	B	?
---	---	---	---	---	---	---	---

t

B	D	C	A	B	A
---	---	---	---	---	---

替换

X

							?
A	B	C	B	D	A	B	

t

B	D	C	A	B	A
---	---	---	---	---	---

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.3 子序列问题 编辑距离

■ 考察末尾字符 删除

s A B C B D A **B**

t B D C A B A

s A B C B D A B

t B D C A B A



$$D[i, j] = D[i-1, j] + 1$$

最优子结构

问题结构分析

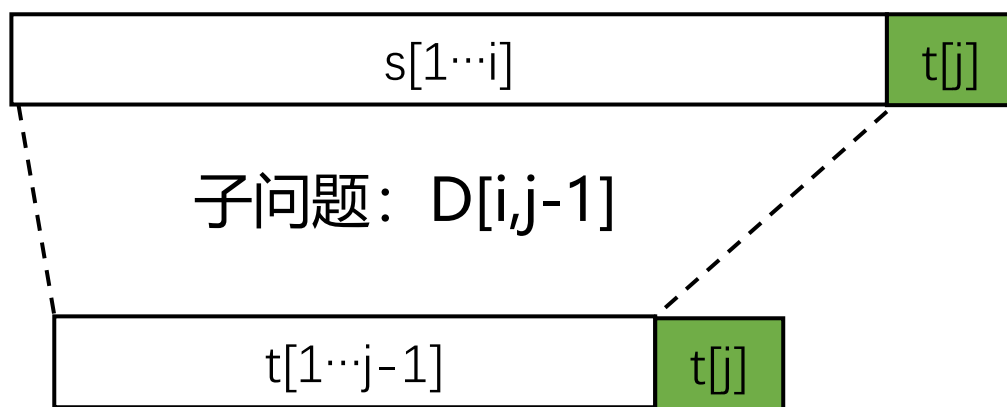
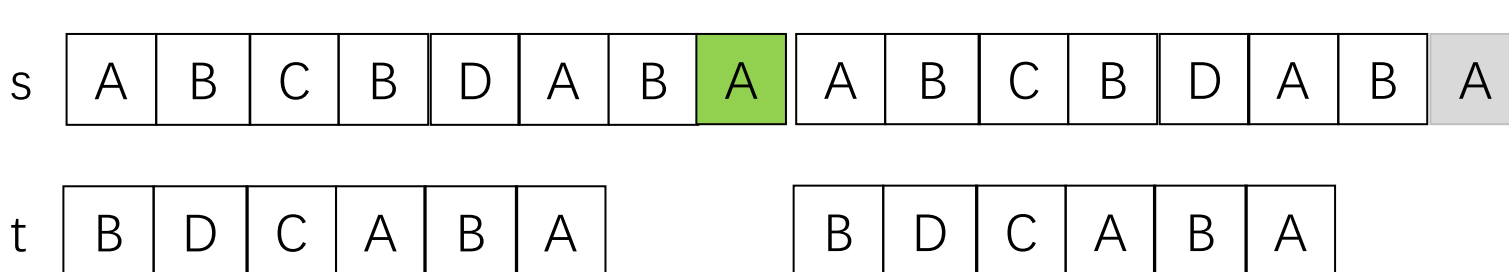
递推关系建立

自底向上计算

最优方案追踪

3.3 子序列问题 编辑距离

■ 考察末尾字符 插入



$$D[i,j] = D[i,j-1] + 1$$

最优子结构

问题结构分析

递推关系建立

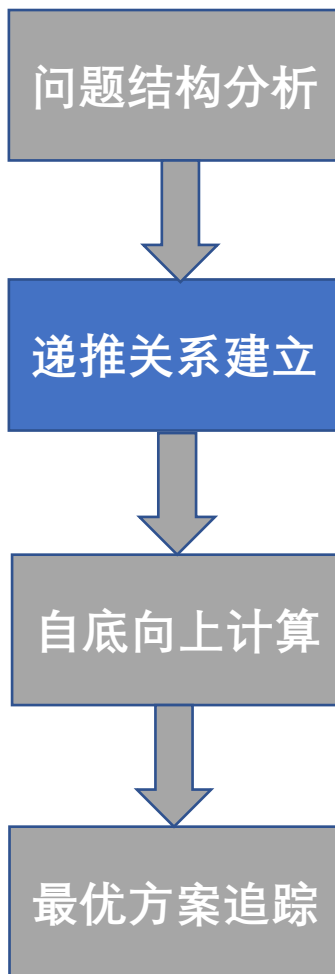
自底向上计算

最优方案追踪

3.3 子序列问题 编辑距离

■ 考察末尾字符 替换

s	A	B	C	B	D	A	B
t	B	D	C	A	B	A	



3.3 子序列问题 编辑距离

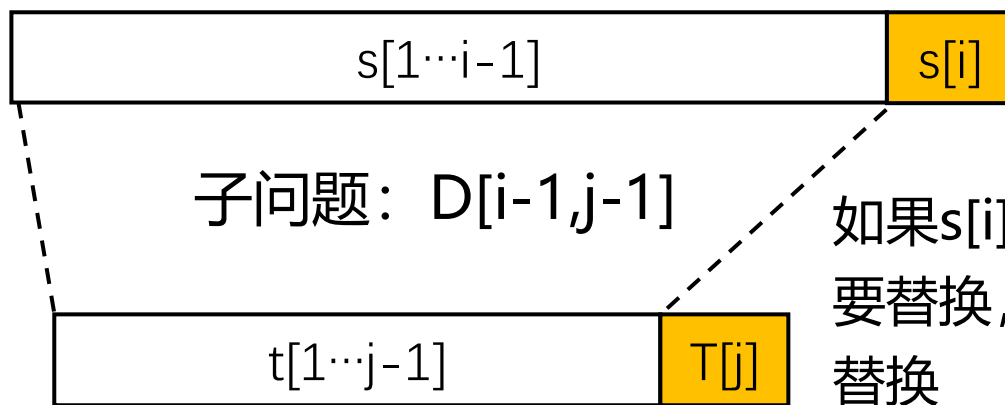
■ 考察末尾字符 替换

s A B C B D A A

t B D C A B A

s A B C B D A A

t B D C A B A



如果 $s[i] = t[j]$, 不需要替换, 否则需要替换

$$D[i, j] = D[i-1, j-1] + \begin{cases} 0, & \text{if } s[i] = t[j] \\ 1, & \text{if } s[i] \neq t[j] \end{cases}$$

最优子结构

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.3 子序列问题 编辑距离

- 综合上面三种方式

$$D[i,j] = \min \begin{cases} D[i-1,j] + 1 \\ D[i,j-1] + 1 \\ D[i-1,j-1] + \begin{cases} 0, & \text{if } s[i]=t[j] \\ 1, & \text{if } s[i]\neq t[j] \end{cases} \end{cases}$$

删除

插入

替换

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

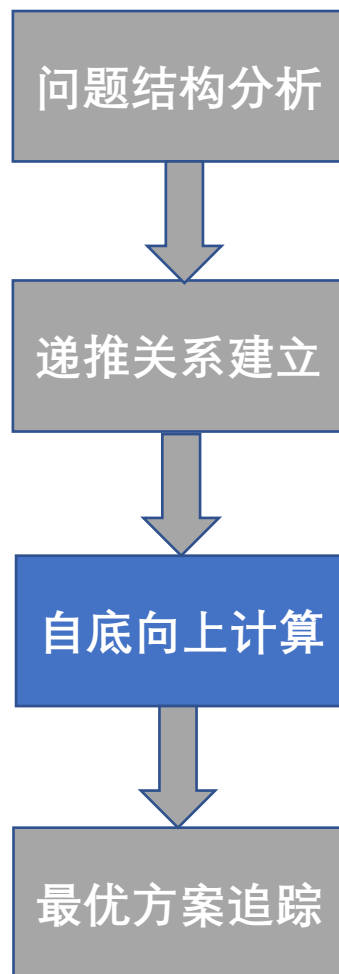
3.3 子序列问题 编辑距离

- 初始化

$D[i,0]=i$ 把长度 i 的串变为空串至少需要 i 次操作(删除)

$D[0,j]=j$ 把空串变为长度 j 的串变为至少需要 j 次操作(插入)

$c[i,j]$	$j=0$	$j=1$	$j=2$	...	$j=m$
$i=0$	0	1	2	...	m
$i=1$	1				
$i=2$	2				
...	...				
$i=n$	n				

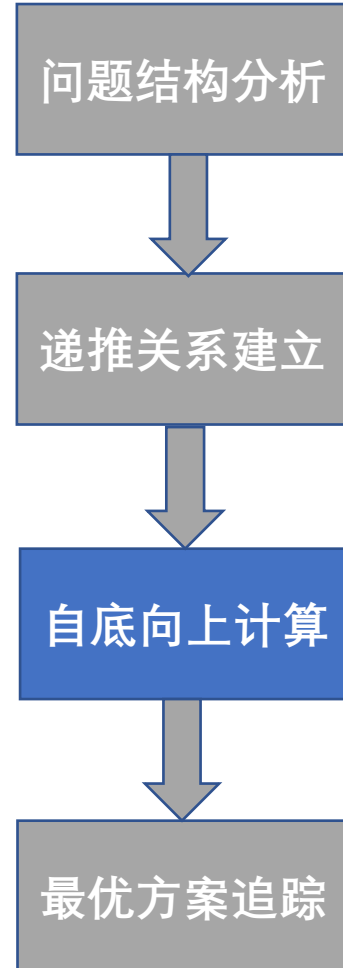


3.3 子序列问题 编辑距离

- 递推公式

$$D[i,j] = \min \begin{cases} D[i-1,j] + 1 & \text{删除} \\ D[i,j-1] + 1 & \text{插入} \\ D[i-1,j-1] + \begin{cases} 0, & \text{if } s[i]=t[j] \\ 1, & \text{if } s[i]\neq t[j] \end{cases} & \text{替换} \end{cases}$$

$c[i,j]$	$j=0$	$j=1$	$j=2$	...	$j=m$
$i=0$	0	1	2	...	m
$i=1$	1	$D[i-1,j-1] + \begin{cases} 0 \\ 1 \end{cases}$	0		
$i=2$	2		1		
...	...				
$i=n$	n		$D[i,j-1] + 1$	$D[i,j]$	



3.3 子序列问题 编辑距离

■ 递推公式

$$D[i,j] = \min \begin{cases} D[i-1,j] + 1 & \text{删除} \\ D[i,j-1] + 1 & \text{插入} \\ D[i-1,j-1] + \begin{cases} 0, & \text{if } s[i]=t[j] \\ 1, & \text{if } s[i]\neq t[j] \end{cases} & \text{替换} \end{cases}$$

$c[i,j]$	$j=0$	$j=1$	$j=2$	$\dots$	$j=m$
$i=0$	0	1	2	$\dots$	m
$i=1$	1				
$i=2$	2	 			
$\dots$	$\dots$	 			
$i=n$	n	 			

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

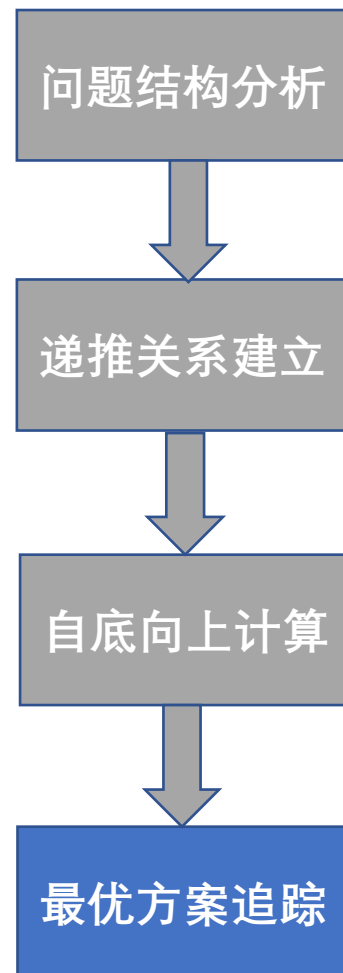
3.3 子序列问题 编辑距离

- 构造追踪数组rec, 记录子问题来源

$$\text{rec}[i,j] = \begin{cases} \text{LU}, D[i-1,j-1] & \text{左上 替换或无操作} \\ \text{U}, D[i-1, j] & \text{上侧 删除s[i]} \\ \text{L}, D[i, j-1] & \text{左侧 插入t[j]} \end{cases}$$

c[i,j]	j=0	j=1	j=2	...	j=m
i=0	0	1	2	...	m
i=1	1			rec[]=LU	
i=2	2				
...	...				
i=n	n		rec[]=U		

rec[]=L



3.3 子序列问题 编辑距离

■ 算法实例

$s[i] \neq t[j]$

	1	2	3	4	5	6	7
s	A	B	C	B	D	A	B
t	B	D	C	A	B	A	

$$D[i,j] = \min \begin{cases} D[i-1,j] + 1 \\ D[i,j-1] + 1 \\ D[i-1,j-1] + \begin{cases} 0, & \text{if } s[i] = t[j] \\ 1, & \text{if } s[i] \neq t[j] \end{cases} \end{cases}$$

$D[]$

	j=	1	2	3	4	5	6
i=0	0	1	2	3	4	5	6
1	1	1					
2	2						
3	3						
4	4						
5	5						
6	6						
7	7						

rec[]

	j=0	1	2	3	4	5	6
i=0		L	L	L	L	L	L
1	U	LU					
2	U						
3	U						
4	U						
5	U						
6	U						
7	U						

3.3 子序列问题 编辑距离

■ 算法实例

	1	2	3	4	5	6	7
s	A	B	C	B	D	A	B
t	B	D	C	A	B	A	

$s[i]=t[j]$

$$D[i,j] = \min \begin{cases} D[i-1,j]+1 \\ D[i,j-1]+1 \\ D[i-1,j-1]+ \begin{cases} 0, & \text{if } s[i]=t[j] \\ 1, & \text{if } s[i] \neq t[j] \end{cases} \end{cases}$$

$D[]$

	j=0	1	2	3	4	5	6
i=0	0	1	2	3	4	5	6
1	1	1	2	3	3	4	5
2	2	1	2	3	4	3	
3	3						
4	4						
5	5						
6	6						
7	7						

$D[i-1,j-1]$

$D[i-1,j]+1$

$D[i-1,j]+1$

rec[]

	j=0	1	2	3	4	5	6
i=0		L	L	L	L	L	L
1	U	LU	LU	LU	LU	L	LU
2	U	LU	LU	LU	LU	LU	
3	U						
4	U						
5	U						
6	U						
7	U						

3.3 子序列问题 编辑距离

■ 算法实例

	1	2	3	4	5	6	7
s	A	B	C	B	D	A	B
t	B	D	C	A	B	A	

$$D[i,j] = \min \begin{cases} D[i-1,j]+1 \\ D[i,j-1]+1 \\ D[i-1,j-1] + \begin{cases} 0, & \text{if } s[i]=t[j] \\ 1, & \text{if } s[i] \neq t[j] \end{cases} \end{cases}$$

D[]

	j=0	1	2	3	4	5	6
i=0	0	1	2	3	4	5	6
1	1	1	2	3	3	4	5
2	2	1	2	3	4	3	4
3	3	2	2	2	3	4	4
4	4	3	3	3	3	3	4
5	5	4	3	4	4	4	4
6	6	5	4	4	4	5	4
7	7	6	5	5	5		

rec[]

	j=0	1	2	3	4	5	6
i=0		L	L	L	L	L	L
1	U	LU	LU	LU	LU	L	LU
2	U	LU	LU	LU	LU	LU	L
3	U	U	LU	LU	L	L	LU
4	U	LU	LU	LU	LU	LU	L
5	U	U	LU	LU	LU	LU	LU
6	U	U	U	LU	LU	LU	LU
7	U	LU	U	LU	LU		

3.3 子序列问题 编辑距离

■ 算法实例

	1	2	3	4	5	6	7
s	A	B	C	B	D	A	B
t	B	D	C	A	B	A	

$D[i,j] = \min \begin{cases} D[i-1,j]+1 \\ D[i,j-1]+1 \\ D[i-1,j-1]+ \begin{cases} 0, \text{ if } s[i]=t[j] \\ 1, \text{ if } s[i]\neq t[j] \end{cases} \end{cases}$

D[]

	j=0	1	2	3	4	5	6
i=0	0	1	2	3	4	5	6
1	1	1	2	3	3	4	5
2	2	1	2	3	4	3	4
3	3	2	2	2	3	4	4
4	4	3	3	3	4	4	4
5	5	4	3	4	4	4	4
6	6	5	4	4	4	5	4
7	7	6	5	5	5	4	5

最优解

rec[]

	j=0	1	2	3	4	5	6
i=0		L	L	L	L	L	L
1	U	LU	LU	LU	LU	L	LU
2	U	LU	LU	LU	LU	LU	L
3	U	U	LU	LU	L	L	LU
4	U	LU	LU	LU	LU	LU	L
5	U	U	LU	LU	LU	LU	LU
6	U	U	U	LU	LU	LU	LU
7	U	LU	U	LU	LU	LU	L

3.3 子序列问题 编辑距离

■ 算法实例

	1	2	3	4	5	6	7
s	A	B	C	B	D	A	B
t	B	D	C	A	B	A	

$$D[i,j] = \min \begin{cases} D[i-1,j] + 1 & \text{删除} \\ D[i,j-1] + 1 & \text{插入} \\ D[i-1,j-1] + \begin{cases} 0, & \text{if } s[i]=t[j] \\ 1, & \text{if } s[i] \neq t[j] \end{cases} & \text{替换} \end{cases}$$

操作
序列

删除A
无操作
删除C
用D替换B
用C替换D
无操作
无操作
插入A

rec[]

	j=0	1	2	3	4	5	6
i=0		L	L	L	L	L	L
1	U	LU	LU	LU	LU	L	LU
2	U	LU	LU	LU	LU	LU	L
3	U	U	LU	LU	L	L	LU
4	U	LU	LU	LU	LU	LU	L
5	U	U	LU	LU	LU	LU	LU
6	U	U	U	LU	LU	LU	LU
7	U	LU	U	LU	LU	LU	L

3.3 子序列问题 编辑距离

■ 伪代码

//动态规划

```
for  $i \leftarrow 1$  to  $n$  do
  for  $j \leftarrow 1$  to  $m$  do
     $c \leftarrow 0$ 
    if  $s_i \neq t_j$  then
       $c \leftarrow 1$ 
    end
     $replace \leftarrow D[i-1, j-1] + c$ 
     $delete \leftarrow D[i-1, j] + 1$ 
     $insert \leftarrow D[i, j-1] + 1$ 
    if  $replace = \min\{delete, insert, replace\}$  then
       $D[i, j] \leftarrow D[i-1, j-1] + c$ 
       $Rec[i, j] \leftarrow "LU"$ 
    end
    else if  $insert = \min\{delete, insert, replace\}$  then
       $D[i, j] \leftarrow D[i, j-1] + 1$ 
       $Rec[i, j] \leftarrow "L"$ 
    end
    else
       $D[i, j] \leftarrow D[i-1, j] + 1$ 
       $Rec[i, j] \leftarrow "U"$ 
    end
  end
end
end
```

$O(m)$ $O(mn)$

时间复杂度: $O(mn)$

3.3 子序列问题 编辑距离

■ 伪代码

Print-MED(*Rec*, *s*, *t*, *i*, *j*)

输入: 矩阵 Rec , 字符串 s, t , 位置索引 i 和 j

输出: 操作序列

```
if  $i = 0$  and  $j = 0$  then  
  | return NULL
```

```
end
```

```
if  $Rec[i, j] = "LU"$  then
```

```
  | Print-MED( $Rec, s, t, i - 1, j - 1$ )
```

```
  | if  $s_i = t_j$  then
```

```
    | print “无需操作”
```

```
  | end
```

```
  | else
```

```
    | print “用 $t_j$ 替换 $s_i$ ”
```

```
  | end
```

```
end
```

```
else if  $Rec[i, j] = "U"$  then
```

```
  | Print-MED( $Rec, s, t, i - 1, j$ )
```

```
  | print “删除 $s_i$ ”
```

```
end
```

```
else
```

```
  | Print-MED( $Rec, s, t, i, j - 1$ )
```

```
  | print “插入 $t_j$ ”
```

```
end
```

递归输出子问题方案

实验3-6：编辑距离

- 给定两个序列X和Y，求两个序列编辑距离。

- 示例 1：

输入：X: "ABCADBCA" Y: "BACDBDA"

输出：

- 要求：

1 完成程序；

2 打印dp数组；

3 打印追踪数组rec[]。

图像压缩

黑白图像存储

像素点灰度值：0~255，为8位二进制数

图像的灰度值序列： $\{p_1, p_2, \dots, p_n\}$ ， p_i 为第*i*个像素点灰度值

图像存储：每个像素的灰度值占8位，总计空间为 $8n$



0	2	15	0	0	11	10	0	0	0	9	9	0	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	115	238	255	244	245	243	250	249	255	222	101	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	95	255	228	255	251	254	211	141	116	127	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
64	41	245	255	217	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	164	6	0
0	13	115	255	255	245	255	182	181	248	252	242	205	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	119	255	245	255	194	9	0
0	111	255	242	255	155	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	115	217	248	253	255	52	4
0	18	148	250	255	247	255	255	255	249	255	240	255	125	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

图像变位压缩的概念

变位压缩存储: 将 $\{p_1, p_2, \dots, p_n\}$ 分成 m 段

$$S_1, S_2, \dots, S_m$$



同一段的像素占用位数相同

第 t 段有 $l[t]$ 个像素，每个占用 $b[t]$ 位

段头: 记录 $l[t]$ (8位) 和 $b[t]$ (3位) 需要 11 位

总位数为

$$b[1] \cdot l[1] + b[2] \cdot l[2] + \dots + b[m] \cdot l[m] + 11m$$

10	12	15
2	1	255
1	1	2
1	1	2

灰度值线性化

图像压缩问题

约束条件：第 t 段像素个数 $l[t] \leq 256$

第 t 段占用空间： $b[t] \times l[t] + 11$

$$b[t] = \left\lceil \log(\max_{p_k \in S_t} p_k + 1) \right\rceil \leq 8$$

问题：给定像素序列 $\{p_1, p_2, \dots, p_n\}$ ，
确定最优分段，即

$$\min_T \left\{ \sum_{t=1}^m (b[t] \times l[t] + 11) \right\},$$

$T = \{S_1, S_2, \dots, S_m\}$ 为分段

实例

灰度值序列

$P=\{10,12,15,255,1,2,1,1,2,2,1,1\}$

分法1: $S_1=\{10, 12, 15\}$, $S_2=\{255\}$,

所需位数:

8	3	4	4	4	8	3	8
---	---	---	---	---	---	---	---

$S_3=\{1, 2, 1, 1, 2, 2, 1, 1\}$

8	3	2	2	2	2	2	2	2	2
---	---	---	---	---	---	---	---	---	---

分法2: $S_1=\{10,12,15,255,1,2,1,1,2,2,1,1\}$

分法3: 分成12组, 每组一个数

存储空间

分法1: $11 \times 3 + 4 \times 3 + 8 \times 1 + 2 \times 8 = 69$

分法2: $11 \times 1 + 8 \times 12 = 107$

分法3: $11 \times 12 + 4 \times 3 + 8 \times 1 + 1 \times 5 + 2 \times 3 = 163$

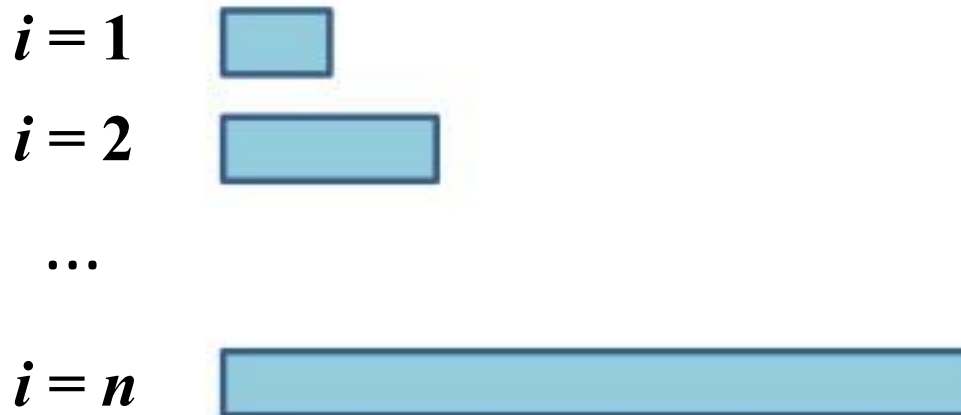
子问题界定与计算顺序

子问题前边界为1，后边界为 i

对应像素序列为 $\langle p_1, p_2, \dots, p_i \rangle$

优化函数值 $S[i]$ 为最优分段存储位数

计算顺序

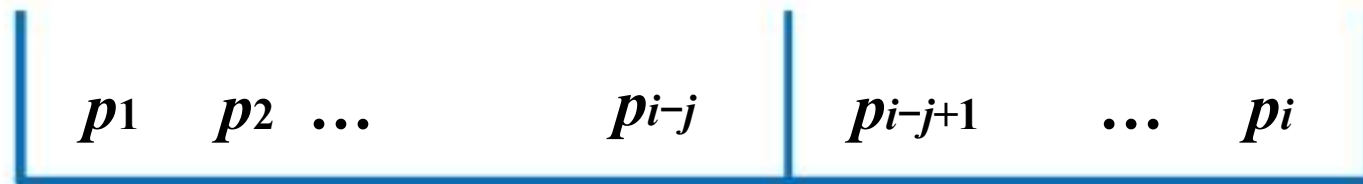


算法设计

递推方程：设 $S[i]$ 是 $\{p_1, p_2, \dots, p_i\}$ 的最优分段需要的存储位数， S_m 是最后分段

$$S[i] = \min_{1 \leq j \leq \min\{i, 256\}} \{ S[i-j] + j \times b[i-j+1, i] \} + 11$$

$$b[i-j+1, i] = \left\lceil \log(\max_{p_k \in S_m} p_k + 1) \right\rceil \leq 8$$



$S[i-j]$ 个位

j 个灰度

$j \times b[i-j+1, i]$ 位

伪码

Compress (P, n)

1. $Lmax \leftarrow 256; header \leftarrow 11; S[0] \leftarrow 0$

2. for $i \leftarrow 1$ to n do

3. $b[i] \leftarrow \text{length}(P[i])$

4. $bmax \leftarrow b[i]$

5. $S[i] \leftarrow S[i-1] + bmax$

6. $l[i] \leftarrow 1$

7. for $j \leftarrow 2$ to $\min\{i, Lmax\}$ do

8. if $bmax < b[i-j+1]$

9. then $bmax \leftarrow b[i-j+1]$

10. if $S[i] > S[i-j] + j * bmax$

11. then $S[i] \leftarrow S[i-j] + j * bmax$

12. $l[i] \leftarrow j$

13. $S[i] \leftarrow S[i] + header$

子问题后
边界 i

最后
段长 j

找到更
好分段

$P = \langle 10, 12, 15, 255, 1, 2 \rangle$.

$S[1]=15, S[2]=19, S[3]=23, S[4]=42, S[5]=50$

$l[1]=1, l[2]=2, l[3]=3, l[4]=1, l[5]=2$

10	12	15	255	1	2
----	----	----	-----	---	---

$S[5]=50$

$1 \times 2 + 11$ 63

10	12	15	255	1	2
----	----	----	-----	---	---

$S[4]=42$

$2 \times 2 + 11$ 57

10	12	15	255	1	2
----	----	----	-----	---	---

$S[3]=23$

$3 \times 8 + 11$ 58

10	12	15	255	1	2
----	----	----	-----	---	---

$S[2]=19$

$4 \times 8 + 11$ 62

10	12	15	255	1	2
----	----	----	-----	---	---

$S[1]=15$

$5 \times 8 + 11$ 66

10	12	15	255	1	2
----	----	----	-----	---	---

$6 \times 8 + 11$

59

追踪解

算法 **Traceback** (n, l)

输入：数组 l

输出：数组 C

1. $j \leftarrow 1$ // j 为正在追踪的段数
2. **while** $n \neq 0$ **do**
3. $C[j] \leftarrow l[n]$ 
4. $n \leftarrow n - l[n]$
5. $j \leftarrow j + 1$

$C[j]$: 从后向前追踪的第 j 段的长度

时间复杂度: $O(n)$